

SIGK - Projekt 5

Stick Animation

Autor: Łukasz Dąbała

1 Wymagania projektu

W ramach projektu należy stworzyć program, który będzie realizował opisane w temacie funkcje. Projekt jest zadaniem zespołowym, gdzie każdy zespół składa się z 2 osób.

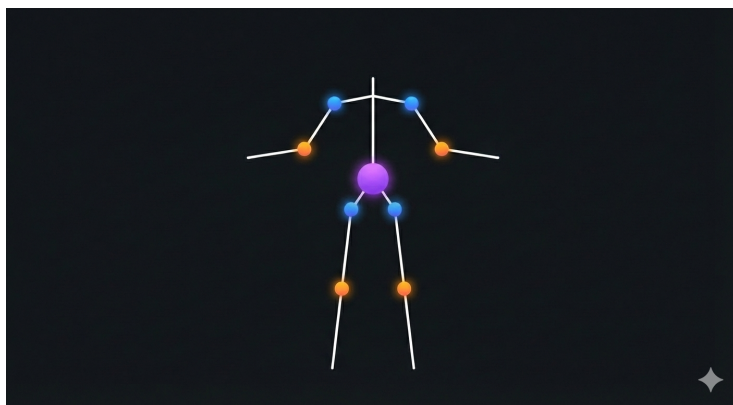
Głównym językiem programowania powinien być język Python wraz z frameworkiem przeznaczonym do sieci neuronowych: Pytorch.

Za projekt można uzyskać maksymalnie $x \times 12p.$, gdzie x to liczba osób w zespole. Każdy z członków zespołu może dostać maksymalnie 12 punktów.

Ocenie w ramach projektu podlegają:

1. Działanie programu - realizacja funkcji (8 p.)
2. Dokumentacja dokonanych eksperymentów oraz wizualizacja wyników (4 p.)

Projekt uznaje się za oddany w momencie prezentacji go prowadzącemu.



Rysunek 1: Przykładowa wizualizacja stickmana

2 Generacja animacji

W ramach projektu należy stworzyć rozwiązanie oparte o sieci dyfuzyjne, które będzie w stanie wygenerować animację tzw. stickmana (patrz rysunek 1). Rodzaj generowanej animacji powinien być dostarczany przez użytkownika w postaci tekstowej. Rozwiązanie powinno umożliwiać generowanie 2 rodzajów animacji:

1. chodu (*ang. walk*)
2. skoku (*ang. jump*)

Datasetem, który można wykorzystać i który zawiera dane dotyczące wymienionych ruchów jest: <https://github.com/una-dinosauria/cmu-mocap/tree/master>. Należy sprawdzić w pliku excelowym, które pliki należą do konkretnego ruchu. Biblioteką, której można użyć do obsługi plików **.bvh**, może być np. *bvhio*.

Wyjściem sieci powinna być animacja szkieletu, który składa się przynajmniej z następujących węzłów: głowa, szyja, miednica, prawe oraz lewe: bark, łokieć, nadgarstek, biodro, kolano, kostka. Daje nam to 15 węzłów szkieletu, który dobrze powinien oddawać strukturę ludzką. Animacja powinna składać się z 48 klatek, co daje nam wyjściowy tensor: $[48, 15, 3]$ tzn. 48 klatek, 15 punktów kluczowych w 3 wymiarach. Po wygenerowaniu szkieletu, należy również dokonać jego wizualizacji tzn. połączyć odpowiednie punkty kluczowe ze sobą i pokazać je na obrazie.

3 Przedstawienie wyników

Oprócz przedstawienia wyników wizualnych tzn. wygenerowanych animacji, należy obliczyć miary takie jak: Frechet Motion Distance (FMD), Mean Per Joint Position Error (MPJPE) oraz wariancję między generowanymi próbkami (tzn. chcemy mieć kreatywny model).

W tabeli powinny znaleźć się wyniki dla każdego rodzaju ruchu:

Ruch	FMD	MPJPE	Var
<i>walk</i>	<wartość>	<wartość>	<wartość>
<i>jump</i>	<wartość>	<wartość>	<wartość>

Tabela 1: Przykład tabeli do przedstawienia wyników

4 Przydatne fragmenty kodu

Generacja animacji na podstawie zadanego tensora.

```
import os

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from enum import IntEnum

class Joint(IntEnum):
    HEAD = 0
    NECK = 1
    PELVIS = 2
    RIGHT_SHOULDER = 3
    RIGHT_ELBOW = 4
    RIGHT_WRIST = 5
    LEFT_SHOULDER = 6
    LEFT_ELBOW = 7
    LEFT_WRIST = 8
    RIGHT_HIP = 9
    RIGHT_KNEE = 10
    RIGHT_ANKLE = 11
    LEFT_HIP = 12
    LEFT_KNEE = 13
    LEFT_ANKLE = 14

JOINT_CONNECTIONS = [
    (Joint.PELVIS, Joint.NECK),
```

```

(Joint.NECK, Joint.HEAD),
(Joint.NECK, Joint.RIGHT_SHOULDER),
(Joint.RIGHT_SHOULDER, Joint.RIGHT_ELBOW),
(Joint.RIGHT_ELBOW, Joint.RIGHT_WRIST),
(Joint.NECK, Joint.LEFT_SHOULDER),
(Joint.LEFT_SHOULDER, Joint.LEFT_ELBOW),
(Joint.LEFT_ELBOW, Joint.LEFT_WRIST),
(Joint.PELVIS, Joint.RIGHT_HIP),
(Joint.RIGHT_HIP, Joint.RIGHT_KNEE),
(Joint.RIGHT_KNEE, Joint.RIGHT_ANKLE),
(Joint.PELVIS, Joint.LEFT_HIP),
(Joint.LEFT_HIP, Joint.LEFT_KNEE),
(Joint.LEFT_KNEE, Joint.LEFT_ANKLE)
]

OUTPUT_PATH = os.path.dirname(__file__)

def animate_skeleton_3d(tensor_data, output_filename=None, fps=24):
    """
    Creates a 3D stickman animation from a coordinate tensor.
    tensor_data: numpy array of shape [T, 15, 3] (T - number of frames)
    Currently supported output type: .gif
    """
    fig = plt.figure(figsize=(8, 8))
    ax = fig.add_subplot(111, projection='3d')

    ax.set_xlim(-1.5, 1.5)
    ax.set_ylim(-1.5, 1.5)
    ax.set_zlim(-1.5, 1.5)

    ax.set_box_aspect([1, 1, 1])
    ax.set_title("3D Stickman Motion Visualization")
    ax.set_xlabel('X Axis')
    ax.set_ylabel('Y Axis')
    ax.set_zlabel('Z Axis')

    points_scatter = ax.scatter([], [], [], c='red', s=40, zorder=3)
    lines = [
        ax.plot([], [], [], c='blue', lw=2, zorder=2)[0]
        for _ in range(len(JOINT_CONNECTIONS))
    ]

    def init():
        points_scatter._offsets3d = ([], [], [])
        for line in lines:
            line.set_data(np.array([]), np.array([]))

```

```

        line.set_3d_properties(np.array([]))
    return [points_scatter] + lines

def update(frame_idx):
    frame_data = tensor_data[frame_idx]

    xs = frame_data[:, 0]
    ys = frame_data[:, 1]
    zs = frame_data[:, 2]
    points_scatter._offsets3d = (xs, ys, zs)

    for i, (start_joint, end_joint) in enumerate(JOINT_CONNECTIONS):
        x_coords = np.array(
            [frame_data[start_joint, 0], frame_data[end_joint, 0]]
        )
        y_coords = np.array(
            [frame_data[start_joint, 1], frame_data[end_joint, 1]]
        )
        z_coords = np.array(
            [frame_data[start_joint, 2], frame_data[end_joint, 2]]
        )

        lines[i].set_data(x_coords, y_coords)
        lines[i].set_3d_properties(z_coords)

    return [points_scatter] + lines

T = tensor_data.shape[0]
anim = animation.FuncAnimation(
    fig, update, frames=T, init_func=init, blit=False, interval=1000 / fps
)
if output_filename:
    anim.save(
        os.path.join(OUTPUT_PATH, output_filename),
        writer='pillow',
        fps=fps
    )
plt.show()

```